

Deadline Based Quantum Inspired Genetic Algorithm for Scheduling Workflows in Cloud-Fog Environment

*Submitted in partial fulfillment of the
requirements for the degree*

of

Masters of Technology

by

Aniruddha Sharma (13011224010)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

TECHNO MAIN SALT LAKE

EM 4/1, SALT LAKE, SECTOR - V, KOLKATA - 700091

CERTIFICATE

This is to certify that the project entitled “Deadline Based Quantum Inspired Genetic Algorithm for Scheduling Workflows in Cloud-Fog Environment” prepared by Aniruddha Sharma (13011224010) of M.Tech (Computer Science & Engineering), Final Year, has been done according to the regulations of the Degree of Masters of Technology in Computer Science & Engineering. The candidates have fulfilled the requirements for the submission of the project report.

It is to be understood that, the undersigned does not necessarily endorse any statement made, opinion expressed or conclusion drawn thereof, but approves the report only for the purpose for which it has been submitted.

(Signature of the Internal Guide)

(Signature of the HOD)

(Signature of the External Examiner)

.....

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

.....

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our project guide in Computer Science and Engineering department. We are extremely thankful for the keen interest he / she took in advising us, for the books and reference materials provided for the moral support extended to us.

Last but not the least we convey our gratitude to all the teachers for providing us the technical skill that will always remain as our asset and to all non-teaching staffs for the gracious hospitality they offered us.

Aniruddha Sharma (13011224010)

Techno Main Salt Lake

Date:

Contents

Abstract	1
1 Introduction	2
2 Proposed Solution	3
2.1 Problem Statement	3
2.1.1 Preliminaries	5
2.2 Proposed DQGA Algorithm	7
2.2.1 Chromosome Representation and Population Initialization . .	7
2.2.2 Chromosome Evaluation	8
2.2.3 Quantum Crossover	9
2.2.4 Quantum Mutation	10
3 Project Planning	12
3.1 Project Schedule & Milestones	12
3.2 Software Development Life Cycle (SDLC): Evolutionary Prototyping	13
3.3 Cost Analysis	14
3.4 Technical Dependencies	14
3.5 Requirement Matrix and Elaboration	16
3.5.1 Functional Requirements	16
3.5.1.1 [FR-01] Workflow Parsing and Dependency Analysis	16
3.5.1.2 [FR-02] Quantum Population Representation	16
3.5.1.3 [FR-03] Algorithm 1: Cumulative Measurement and State Collapse	16
3.5.1.4 [FR-04] Makespan Calculation Engine	16
3.5.1.5 [FR-05] Population Update via Quantum Rotation Gates	16
3.5.1.6 [FR-06] Task-Aware Crossover Operations	17
3.5.2 Non-Functional Requirements	17
3.5.2.1 [NFR-01] Convergence Efficiency	17
3.5.2.2 [NFR-02] System Scalability and Robustness	17
3.5.2.3 [NFR-03] Statistical Stability and Accuracy	17
3.5.3 Interface Requirements	17
3.5.3.1 [IR-01] Graphical Schedule Visualization	17
3.5.3.2 [IR-02] Data Input Support	17
3.5.3.3 [IR-03] Real-Time Execution Telemetry	18
4 Design	18
4.1 Technical Environment	18
4.1.1 Hardware Requirements	18
4.1.2 Operating System	18
4.1.3 Software Environment	19
4.2 Detailed Design	19
4.2.1 Workflow Ingestion and DAG Parsing Module [FR-01]	20
4.2.2 Quantum Engine and Cumulative Measurement [FR-02, FR-03]	20

DQGA for scheduling workflows in cloud fog environment

4.2.3	Makespan Simulation and Fitness Module [FR-04]	20
4.2.4	Genetic Operator and Update Module [FR-05, FR-06]	20
4.2.5	Visualization and Interface Module [IR-01]	21
5	Implementation	22
5.1	Workflow Ingestion & DAG Construction Module	22
5.2	Quantum Population & Q-bit Initialization Module	22
5.3	Measurement & Fitness Evaluation Subsystem	22
5.4	Quantum Rotation & Genetic Operator Module	23
5.5	System Installation Steps	23
5.5.1	Prerequisites	23
6	Test Results and Analysis	23
6.0.1	Performance Metrics	25
6.0.2	Comparison of DQGA with other Classical Algorithms	25
7	Conclusion	29
	References	30
A	Prototype Features	36
B	Filtered Source Code Showing Prototype Features	37
B.1	Workflow Input Processing	37
B.2	Task Graph Construction and DAG Visualization	38
B.3	Quantum-Inspired Cumulative Measurement	39
B.4	Adaptive Quantum Rotation Gate	40
B.5	Makespan and Cost Evaluation	41
C	Software Requirements	43
D	Hardware Requirements	43
E	Compilation and Execution Steps	43
F	Project Directory Structure	44

Abstract

The advent of smart services and IoT devices generates substantial amounts of data. Processing this data requires cloud access, which causes network congestion and transmission delays. Whereas the pervasive deployment of fog nodes enables low-latency processing at the network edge. Because fog nodes have constrained computing resources, data are transferred to the cloud for further processing. Optimizing workflow task allocation and execution in a cloud-fog architecture is a challenging and unpredictable process. The problem of scheduling workflows under the defined deadline requirements while reducing execution cost is a complex problem. Therefore, in this work, a Deadline-Based Quantum-Inspired Genetic Algorithm (DQGA) is designed to offload tasks to the appropriate computational nodes. DQGA is inspired by quantum computing and the genetic algorithm. The genetic operators: selection, crossover, and mutation are combined with quantum operators such as measure, interference, and rotation. The genetic algorithm introduces a new population with the best fitness value, generates near-optimal solutions for NP-hard problems, and is flexible and adaptable. Quantum computing is used for massive parallelism, robustness, rapid convergence, and the maintenance of the diversity of the new population. Further, the DQGA has a new measure function and crossover technique to enhance the efficacy of the proposed work. The simulation results depict that DQGA achieves better performance in terms of minimum execution cost and execution time.

1 Introduction

The proliferation of cloud and fog computing technologies enhances the existing Internet of Things architectures. The IoT enables billions of physical devices to connect and exchange information, powering a wide range of applications. These applications generate vast amounts of data that require fast processing and analysis. Over successive decades, data-sensitive applications have been analyzed, stored, and computed on centralized cloud resources. However, applications that access cloud resources can cause network congestion and latency. The centralized cloud resources fail to meet the latency constraints imposed by latency-sensitive IoT applications, necessitating more distributed solutions [1]. Consequently, a suitable computing architecture for latency-sensitive applications is required. Fog computing architecture extends cloud architecture, improving the performance of IoT applications [2], [3]. A fog computing architecture provides cloud-like services at the network edge. Fog nodes are deployed near IoT devices to handle time-sensitive applications. Whereas, the applications that are computationally intensive are offloaded to the cloud resources for storage and computation [4], [5]. Generally, these IoT applications are mainly composed of several dependent and independent tasks represented as workflows. These workflows can be modeled as a DAG (Directed Acyclic Graph) where each task pair comprises potential dependencies [6]. Workflow Scheduling involves the effective allocation and orchestration of multiple tasks across different fog and cloud resources to satisfy user-defined constraints [7]. Mapping tasks to available resources is crucial to optimize the makespan and cost under the defined deadline constraints, which contributes to an NP Complete problem [8]. In existing studies, researchers have proposed several heuristics and meta-heuristic techniques to overcome the challenges of the workflow scheduling problem [9], [10], [11], [12]. Meta-heuristic algorithms are being widely used because of their simple implementation and high exploration capability [13]. These algorithms are efficient at optimizing multiple parameters simultaneously in terms of time, cost, and energy [14]. A genetic algorithm is a significant metaheuristic that has been widely accepted in workflow scheduling problems [15], [16]. The effectiveness of GA in terms of robustness, adaptability, and global search capabilities has made it popular among researchers. Further, many other algorithms, such as PSO [17], SA [18], and ACO [19], have contributed significantly to the workflow scheduling problem [20], [21]. These algorithms work among the set of possible solutions and

employ different rules to select the optimal solution. However, workflow scheduling is a challenging problem, and there exist several issues in terms of local convergence, improper balance between exploitation and exploration, low convergence speed, and underutilization of search space [22]. To overcome these challenges and optimize workflow scheduling algorithm performance, a new deadline-constrained quantum-inspired genetic algorithm for scheduling in a cloud-fog environment has been proposed. The proposed DQGA algorithm aims to minimize execution cost while satisfying the deadline constraints. DQGA is inspired by the principle of quantum computing. The fundamental properties of quantum computing in terms of superposition, entanglement, and rotation enhance the working of DQGA. The proposed algorithm is characterized by the generation of individual chromosomes, the evaluation function, and the dynamic population. The algorithm further combines the selection, crossover, and mutation mechanisms with different quantum operators. Further, a new cumulative-based measure function is used, where the quantum bits are used as a probabilistic representation rather than classical bits, because they explore the search space more extensively. In addition to this, a new crossover technique named Quantum Adaptive Clustered Crossover with Rotation (QACCR) has been proposed. QACCR employs a cluster-preservation crossover and an adaptive quantum rotation mechanism to preserve population quality, resulting in better solutions, faster convergence, and more robust results. Extensive experiments are performed to demonstrate the effectiveness of the DQGA algorithm.

2 Proposed Solution

2.1 Problem Statement

The objective of the proposed methodology is to reduce the total execution cost of all tasks scheduled on the different cloud and fog nodes such that the assigned deadline constraints are minimized. It can be represented as:

$$\text{Minimize: } tec_w$$

$$\text{Subject to: } m_w \leq D_w$$

Where tec_w represents the workflow execution cost and m_w is the makespan, i.e., the total execution time of the workflow. It includes the execution time and the

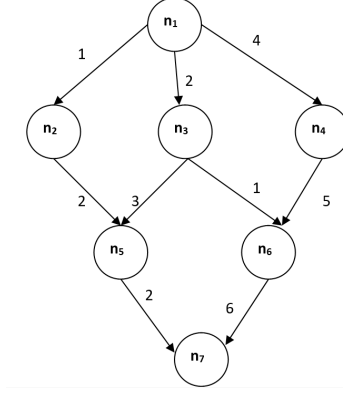


Figure 1: DAG

communication time of the task. The execution time of a task n_j is depicted as $ec_{(n_j)}$ that can be derived from the computation load of n_j (x_{n_j}) and the computing capacity of the computing nodes (γ_{r_k}), where r_k is the k^{th} resource of type r . r can be a fog or a cloud node. The ec_{n_j, r_k} is computed from eq.1

$$ec_{n_j, r_k} = \frac{x_{n_j}}{\gamma_{r_k}} \quad (1)$$

In Figure 1, it is shown that the tasks depend on each other, i.e., if n_i finishes, then only n_j is executed. The data from task n_i is transferred to its successor task n_j , and therefore, the required communication time must be considered. If parent and child tasks are allocated on the same node, then the communication time is assumed to be zero. If they are scheduled on a different computational node, the amount of data transferred and the bandwidth of the computational node are taken into consideration, as shown in eq.2.

$$tt_{n_i, n_j} = \begin{cases} 0, & \text{if } r_i = r_j, \\ \frac{dt(n_i, n_j)}{\tau_{i,j}}, & \text{if } r_i \neq r_j \end{cases} \quad (2)$$

Where $dt(n_i, n_j)$ represents the time required to transfer data from task n_i to task n_j and $\tau_{i,j}$ is the data transfer rate. A task assigned to a computational node is associated with two execution parameters namely the start time (st) and finish time (ft). The status of a task depends on the finish time of its ancestor node, the data transfer time between its predecessors, and the availability of the computational node on which it is to be executed. Start time of task n_j on r_k (st_{n_j, r_k}) can be computed

from eq.3.

$$st_{n_j, r_k} = \begin{cases} 0, & \text{if } n_j == n_{entry}, \\ \max\{avail(r_k)\}, \max_{n_i \in pred_j} \{ft_{n_i} + tt_{n_i, n_j}\} & \end{cases} \quad (3)$$

Where $avail(r_k)$ is the time when the k^{th} resource becomes available for the execution of the task n_j . $pred_j$ is the immediate predecessor of task n_j , ft_{n_i} denotes the finish time of task n_i computed from eq.4 and tt_{n_i, n_j} represents the time incurred to transmit data from the task n_i to the task n_j .

$$ft_{n_j, r_k} = st_{n_j, r_k} + ec_{n_j, r_k} \quad (4)$$

Afterward, the total completion time, i.e., makespan (m_w), can be evaluated from eq.5.

$$m_w = \max\{ft_{n_j}, n_j \in N\} \quad (5)$$

The utilization of computational nodes, whether cloud or fog, incurs charges for their usage. Therefore, a charging model from the work [23] has been adopted here. The computational cost (cmp_w) for a workflow is derived from the eq.6, where cp_{r_k} is the unit price of computation for k^{th} resource of type r . Whereas, the communication cost (ccp_w) of a workflow is the total cost of data transfer when tasks are scheduled on different nodes obtained from eq.7 and ct_{r_k} is the cost incurred per unit of data communication on resource r_k . The total execution cost of a workflow can be obtained from the eq.8.

$$cmp_w = \sum_{i=j}^N (ft_{n_j} - st_{n_j}) * cp_{r_k} \quad (6)$$

$$ccp_w = \sum_{k=1}^R tt_{n_i, n_j} * ct_{r_k} \quad (7)$$

$$tec_w = cmp_w + ccp_w \quad (8)$$

2.1.1 Preliminaries

This section provides some preliminary ideas on quantum computing indispensable to understanding the operation of the proposed DQGA algorithm. The DQGA algorithm combines the benefits of the genetic algorithm and quantum computing together to solve the optimization problem. The widespread use of the genetic algorithm is

attributed to its problem-independent nature, low complexity, and ability to generate near-optimal results. The genetic algorithm produces individuals using different operators: selection, crossover, and mutation. These operators generate the best individuals with higher fitness, thereby preserving the high-quality individual population [24], [25]. However, despite its many advantages, GA is time-consuming and exhibits slow convergence. It tends to get stuck in local optima, and therefore, many researchers have explored hybrid models to improve the performance of existing solutions. For this reason, the proposed approach incorporates the concepts of both quantum computing and genetic algorithms. The DQGA algorithm is inspired by the quantum bits and quantum principles, such as superposition, rotation and entanglement. The distinct properties of quantum modeling enable efficient exploration of the search space, thereby facilitating the identification of optimal solutions. Quantum-inspired algorithms can effectively explore multiple potential solutions simultaneously, leading to more efficient utilization of the search space than existing approaches. This is because quantum algorithms tend to exploit and explore the search space in parallel and probabilistic ways. Before explaining DQGA in detail, the basics of quantum computing are addressed briefly. In the classical way, the information is always stored in a binary format of ‘0’ and ‘1’. In quantum computing, quantum bits (Qubits) are in a state of either 0 or 1, or a superposition of both [26]. A qubit state can be denoted as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad (9)$$

Where α and β are complex numbers that define the probability amplitude of the corresponding states and must satisfy the condition defined in eq.10, where α^2 and β^2 indicate the likelihood of the qubit collapsing to state 0 and 1, respectively, upon measurement.

$$\alpha^2 + \beta^2 = 1 \quad (10)$$

The qubit state can change its position with the help of quantum gates. There are numerous quantum gates defined in quantum computing including the Hadamard gate, the rotation gate, the NOT gate, etc. [27]. If the system consists of n quantum bits, then there can be 2^n quantum states at the same time. However, if the system is measured, it collapses to a single state. Leveraging concepts of quantum computing, DQGA is designed with a Q-bit representation, a novel measurement function, a new crossover technique, and combined quantum and genetic operators.

2.2 Proposed DQGA Algorithm

This section introduces the proposed DQGA, Deadline-Based Quantum-Inspired Genetic Algorithm for scheduling workflows across available cloud and fog nodes. The algorithm aims to reduce the execution cost subject to the deadline constraints. The algorithm comprises a quantum model that addresses premature convergence and guides the search towards optimal or near-optimal solutions. On the other hand, genetic operators incorporate individuals with higher fitness values, select the best individuals, and balance search diversity through the novel crossover technique. The working of the proposed algorithm has been shown in Algorithm

2.2.1 Chromosome Representation and Population Initialization

In the genetic algorithm, the individual chromosomes represent a complete solution. Generally, these chromosomes are encoded as a string of genes, which can be in any format, such as bits, characters, integers, and many more. However, the proposed approach is based on quantum-inspired GA; therefore, quantum chromosomes are represented as a string of l qubits, where each qubit represents the assignment of a task to the corresponding computational nodes shown in Figure 2. The qubit value denotes the specific computation node onto which the task is assigned. In the designed algorithm, these values are altered in each iteration, where different combinations of tasks are assigned to nodes and evaluated. The chromosome population can be represented by a set of qubit chromosomes denoted by P_t as shown in eq where P_t represents the population at iteration t , n is the population size and ch_m^t is the m_{th} chromosome of iteration t , belonging to $m = 1, 2, \dots, n$ shown in Figure2.

Initially, for each chromosome, the coefficient of qubits (α and β) is assigned randomly refer Figure 2.

Task (n_1)	α_1	β_1	α_2	β_2	α_3	β_3
	0.3745	0.9272	0.9507	0.3092	0.7319	0.6814
Task (n_2)	α_4	β_4	α_5	β_5	α_6	β_6
	0.5987	0.8009	0.1560	0.9877	0.1559	0.9877
Task (n_N)	α_7	β_7	α_8	β_8	α_l	β_l
	0.6011	0.7992	0.8662	0.4998	0.0581	0.9983

Figure 2: Chromosome Representation

Algorithm 1 Deadline-Based Quantum-Inspired Genetic Algorithm (DQGA)

Require: Set of tasks N , set of cloud and fog resources (R), P , max_{iter}

Ensure: Optimal schedule

- 1: Initialize Population (P) using eq.22
 - 2: Evaluate each chromosome $ch \in P$ using Algorithm 2 and eq.11
 - 3: Store the best chromosome ch in $best(t)$
 - 4: $t = 0$
 - 5: **while** $t \leq max_{iter}$ **do**
 - 6: **for** each $ch \in P_t$ **do**
 - 7: Select parent chromosomes ch from parent P_t
 - 8: Apply crossover operator to selected parents from Algorithm 3
 - 9: Perform mutation operator to offspring
 - 10: Evaluate fitness of new population
 - 11: Store best chromosome ch in $best(t)$
 - 12: Generate new population $P(t + 1)$
 - 13: $t = t + 1$
 - 14: **end for**
 - 15: **end while**
 - 16: **return** Best Chromosome
-

2.2.2 Chromosome Evaluation

A measure function named: the Cumulative Based Measure Function (CMF) shown in Algorithm 2 has been proposed to measure the values of each chromosome at iteration t and then generate a binary representation for it. The binary string denotes the computational node on which it is to be scheduled. Figure 3 generates the DAG mapping schedule using the cumulative measure function from Algorithm 2. This

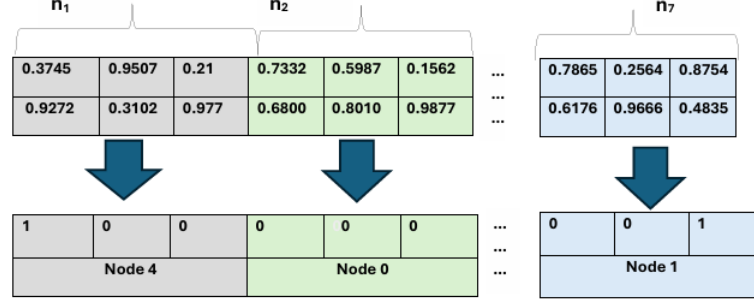


Figure 4: Measured Chromosome

Figure 3: Measured Chromosome

mapping shows which task has been assigned to which computational nodes. Afterward, the makespan and execution cost are computed using eq.5 and eq.8. On the basis of this, the fitness of the individual chromosome is determined. This work considers only one QoS constraint, namely the deadline constraint. The fitness of the individual chromosome is obtained from the eq.11, where the chromosome that has the minimum total cost of execution without deadline violation has the highest fitness value. Then, if both chromosomes meet the deadline constraints, the schedule with the lower total execution cost will have higher preference. Further, in cases where there is one viable and one non-viable schedule, the schedule that satisfies the deadline constraint will be selected. Whereas, when there is no viable schedule, the schedule with the minimum deadline violation is considered [28].

$$f_{ch} = tec_w \quad (11)$$

2.2.3 Quantum Crossover

In DQGA, to maintain population diversity, a new algorithm named: Quantum Adaptive Clustered Crossover with Rotation (QACCR) has been proposed. The proposed crossover mechanism recombines the characteristics of two individuals to generate an optimal solution. The binary tournament selection method is used to select the individual chromosome, and then crossover is performed. Due to minimum selection pressure, eliminating the need to sort the population, and reduced bias towards selecting only the fittest individuals, the binary tournament selection method is preferred.

For crossover, the two best parents are selected to produce offspring, as shown in Algorithm 3. In step 1, the fitness-based weight (η) for both parents is evaluated using eq.12. Afterward, the tasks of each chromosome are clustered according to their level in step 2. Tasks in a DAG with the same level are grouped into a cluster. In step 3, the cluster feasible cost cfc of each cluster is obtained using eq.13, and then the cfc_k is normalized using eq.14. The score of each group is then evaluated using eq.15 and eq.16. Cluster Crossover Probability determines the probability of crossover to happen (P_k), which is evaluated using $ncfc_k$ and ns_k in eq.17. Afterward, a random number between 0 and 1 is generated and compared with P_k ; if the value of P_k is greater than rn , then the variable M_k becomes 1; otherwise, it is 0. If M_k is 1, then that cluster is selected for crossover; otherwise, the child inherits from the competent parent. The selected cluster for crossover is again refined using eq.19, eq.20 and eq.21.

2.2.4 Quantum Mutation

DQGA is inspired by the work of [29] for introducing diversity in the population. The same mutation mechanism has been applied with a probability of 0.01, where each qubit position may change according to the predefined probability.

$$\eta = \frac{f_{ch_i}}{f_{ch_i} + f_{ch_j}} \quad (12)$$

$$cfc_k = \min \sum_{n_j \in C_k} cost_{n_j} \quad (13)$$

$$\text{subject to } m_w \leq D_w$$

$$ncfc_k = \frac{cfc_k}{cfc_{max}} \quad (14)$$

$$s_k = \min \sum_{n_j \in C_k} cost_{n_j} \quad (15)$$

$$ns_k = \frac{s_k}{\max(s_1, s_2, \dots, s_y)} \quad (16)$$

$$P_k = \lambda(1 - ncfc_k) + (1 - \lambda)(1 - ns_k) \quad (17)$$

$$M_k = \begin{cases} 1, & r < P_k \\ 0, & \text{otherwise} \end{cases} \quad (18)$$

$$|\psi_{C_k}^{ch'}\rangle = \eta |\psi_{C_k}^{ch_i}\rangle + (1 - \eta) |\psi_{C_k}^{ch_j}\rangle \quad (19)$$

$$a_y(\theta) = \begin{bmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{bmatrix} \quad (20)$$

$$|\psi_i^{C_k, ch'}\rangle = a_y(\theta_i) |\psi_{C_k}^{ch'}\rangle \quad (21)$$

Algorithm 2 Cumulative Based Measure Function

Require: Basis state of n bit Qubits, N : Total no. of task, max_{iter} : maximum number of iteration, Chromosome Population P

Ensure: Collapsed state in terms of classical string 0/1

```

1: for  $ch \in P$  do
2:   for  $\forall n_j \in N$  do
3:     Generate a quantum inspired amplitude vector for  $l$  bit qubit state  $\gamma =$ 
        $[\gamma_0, \gamma_1, \gamma_2, \dots, \gamma_{2l-1}]$ , where  $\gamma_k \in [0, 1]$ , where  $k = 0, 1, 2, \dots, 2l - 1$  and  $\sum_k \gamma_k^2 = 1$ 
4:     Convert amplitude into probabilities  $\alpha_{j,k} = |\gamma_{j,k}^2|$ 
5:     Find cumulative frequency  $cf_k$  of each state
6:     for  $int\ t=0; t \leq max_{iter}; t++$  do
7:       Generate a random number  $rn_t[0,1]$ 
8:       Find first basis state  $rn_t < cf_{s_j}$ 
9:     end for
10:    Select the collapsed state having maximum number of count
11:  end for
12: end for
13: return Resource Mapped Schedules

```

Algorithm 3 Quantum Adaptive Clustered Crossover with Rotation (QACCR)

Require: Parent Chromosome (ch_i, ch_j), Fitness of parent chromosome (f_{ch_i}, f_{ch_j}),
 k^{th} cluster: C_k , Total Cluster: C_y

Ensure: Child Chromosome (ch')

- 1: Find the fitness-based weight(η) for both the parents using eq.12
- 2: Find task clusters for the given Chromosomes, $C_{x,ch_i} = C_{1,ch_i}, C_{2,ch_i}, C_{3,ch_i} \dots C_{y,ch_i}$
- 3: **for** each $C_k \in C_y$ **do**
- 4: Compute cf_c for each cluster using eq.13 and eq.14
- 5: Evaluate the ns_k score of cluster using eq.15 eq.16
- 6: Compute Cluster Crossover Probability (P_k) for each cluster using eq.17
- 7: Evaluate M_k using eq.18
- 8: **if** $M_k == 1$ **then**
- 9: Apply crossover on cluster C_k using eq.19
- 10: Refine each qubit of the cluster C_k for further improvement using eq.20 and eq.21
- 11: **else**
- 12: Inherit best Parent genes
- 13: **end if**
- 14: **end for**
- 15: **return** Child Chromosome ch'

$$P_t = \{ch_1^t, ch_2^t, ch_3^t, \dots ch_n^t\} \quad (22)$$

3 Project Planning

This project followed a **Prototypive Evolutionary Model**, which allowed for iterative refinement of the Quantum-Inspired Genetic Algorithm. This model was chosen because the stochastic nature of quantum heuristics requires constant testing and parameter tuning (such as $\Delta\theta$ and P_m) before final deployment.

3.1 Project Schedule & Milestones

The project was executed over a 24-week timeline (approximately 6 months), divided into six major phases.

Phase	Task Name	Duration	Dependency	Key Milestone
P1	Literature Review & Problem Definition	4 Weeks	None	Finalized Research Gap
P2	Environment Setup & Dataset Acquisition	2 Weeks	P1	Validated Cyber-shake/Montage Parsers
P3	Core QIGA Engine Development	6 Weeks	P2	Successful Bit-wise QGA Convergence
P4	Algorithm 1: Cumulative Measurement	4 Weeks	P3	Integration of Statistical Sampling
P5	Comparative Analysis & Optimization	4 Weeks	P4	Performance Benchmarking Graphs
P6	Documentation & Thesis Writing	4 Weeks	P5	Final Project Report Submission

Table 1: Project Schedule and Milestones

3.2 Software Development Life Cycle (SDLC): Evolutionary Prototyping

We adopted the **Evolutionary Prototyping Model** because the optimal quantum rotation gate logic could not be fully defined upfront.

- **Initial Prototype:** Developed a standard bit-wise QGA to verify DAG scheduling logic.
- **Refinement:** Integrated the Cumulative Frequency (CF) logic to stabilize results.
- **Iteration:** Added Task-Aware Crossover to resolve the “broken node ID” issue identified during testing.
- **Final System:** The validated model was scaled to larger datasets (1000+ tasks).

3.3 Cost Analysis

Since this is a software-based research project, the costs are primarily associated with computational resources and human capital.

Category	Item Description	Estimated Cost (INR)
Hardware	High-performance workstation (6-core CPU, 16GB RAM)	65,000 (Existing)
Cloud	Azure/AWS Free Tier (for benchmarking validation)	0
Software	Python, VS Code, LaTeX, MS Project	Open Source / Student License
Man-Hours	Developer/Researcher (approx. 480 hours)	Research-based (N/A)
Total		65,000

Table 2: Cost Analysis

3.4 Technical Dependencies

- **Data Dependency:** The `sqga()` function requires the `load_etc()` and `load_tt()` modules to provide valid matrices.
- **Logic Dependency:** The `cumulative_measure` function depends on the `bits_per_node` calculation to ensure proper alignment with the qubit string length.
- **Environment Dependency:** Requires libraries such as `matplotlib` and `networkx` for visualization and reporting.

Table 3: Requirement Matrix

ID	Category	Description	Priority
FR-01	Functional	The system must parse Directed Acyclic Graph (DAG) files (XML/TXT) to identify task dependencies.	High
FR-02	Functional	The system must implement Q-bit representation using (α, β) probability amplitudes for task mapping.	High
FR-03	Functional	Algorithm 1 Integration: The system must perform cumulative frequency sampling and majority voting for state collapse.	High
FR-04	Functional	The system must calculate Makespan using Earliest Start Time (EST) and Earliest Finish Time (EFT) logic.	Medium
FR-05	Functional	The engine must apply Quantum Rotation Gates ($\Delta\theta$) to update the population toward the global best.	Medium
FR-06	Non-Functional	The system must implement "Task-Aware" crossover to prevent invalid node ID generation.	High
NFR-01	Non-Functional	Convergence Speed: The system should reach an optimal makespan within 50 generations for 100-task workflows.	Medium
NFR-02	Non-Functional	Scalability: The scheduler must handle up to 1000 tasks (e.g., <i>Cybershake₁₀₀₁</i>) <i>without memory overflow</i> .	High
NFR-03	Non-Functional	Accuracy: The cumulative measurement must show a lower standard deviation in makespan compared to bit-wise QGA.	High
IR-01	Interface	The software must provide a graphical Gantt chart output using Matplotlib for schedule visualization.	Medium
IR-02	Interface	The system must support CSV/TXT input for Expected Time to Compute (ETC) and Transfer Time (TT) matrices.	High
IR-03	Interface	The console must output real-time "Best Makespan" updates per generation for user monitoring.	Low

3.5 Requirement Matrix and Elaboration

This section details the specific functional, non-functional, and interface capabilities necessitated by the QIGA-CM framework. The focus remains on “*what*” the system must perform to achieve optimal cloud scheduling.

3.5.1 Functional Requirements

3.5.1.1 [FR-01] Workflow Parsing and Dependency Analysis The system must possess a robust parser capable of ingesting Directed Acyclic Graph (DAG) structures from standardized formats (XML/TXT). It is required to extract task execution requirements, identify parent-child dependencies, and calculate the data transfer overhead between tasks.

3.5.1.2 [FR-02] Quantum Population Representation The system is required to model the task-to-node mapping using quantum probability amplitudes. Each chromosome must store Q-bits represented by (α, β) pairs, allowing the system to maintain a superposition of multiple potential scheduling solutions simultaneously.

3.5.1.3 [FR-03] Algorithm 1: Cumulative Measurement and State Collapse A specialized measurement module is needed to implement Cumulative Frequency (CF) sampling. Instead of a single random collapse, the system must perform multiple internal iterations (measurements) to build a probability distribution, followed by a Majority Vote to determine the final discrete resource ID.

3.5.1.4 [FR-04] Makespan Calculation Engine The system must include a scheduling logic engine that computes the Earliest Start Time (EST) and Earliest Finish Time (EFT) for every task. It must account for both the execution time (from the ETC matrix) and the communication delay (from the Transfer Time matrix) to determine the overall makespan.

3.5.1.5 [FR-05] Population Update via Quantum Rotation Gates The system requires an update mechanism using Quantum Rotation Gates ($U(\Delta\theta)$). This module must dynamically adjust the rotation angle based on the fitness of individuals relative to the global best solution to guide the population toward convergence.

3.5.1.6 [FR-06] Task-Aware Crossover Operations The system must support a specialized crossover operator that respects task boundaries. It is needed to ensure that when two chromosomes exchange information, the bits representing a specific task are moved as a cohesive unit to prevent the generation of invalid node assignments.

3.5.2 Non-Functional Requirements

3.5.2.1 [NFR-01] Convergence Efficiency The algorithm is required to demonstrate rapid convergence. It should be capable of identifying an optimal or near-optimal makespan within a maximum of 50 generations for medium-scale workflows (approximately 100 tasks).

3.5.2.2 [NFR-02] System Scalability and Robustness The implementation must be scalable to handle high-complexity scientific workflows. The system is required to process up to 1000 tasks (e.g., Cybershake_1001) without encountering memory leaks or exceeding standard computational time limits.

3.5.2.3 [NFR-03] Statistical Stability and Accuracy The system must provide higher result consistency than standard QGAs. The cumulative measurement approach is needed to yield a lower variance (standard deviation) in makespan results across multiple execution runs.

3.5.3 Interface Requirements

3.5.3.1 [IR-01] Graphical Schedule Visualization The system must provide a visual output in the form of a Gantt Chart. This interface is needed to display task assignments across different virtual machines and highlight periods of resource idleness or bottlenecks.

3.5.3.2 [IR-02] Data Input Support The system requires a standardized interface to load external data. It must support the import of Expected Time to Compute (ETC) and Transfer Time (TT) matrices in CSV or delimited TXT formats to facilitate testing across different heterogeneous environments.

3.5.3.3 [IR-03] Real-Time Execution Telemetry A console-based monitoring interface is needed. The system must output the “*Best Makespan*” and “*Average Fitness*” for every generation, allowing the user to track the optimization progress in real-time.

4 Design

4.1 Technical Environment

The development and execution environment for the **QIGA-CM framework** is optimized for high-performance mathematical simulations and evolutionary computing. The following specifications ensure the stability and reproducibility of the scheduling results.

4.1.1 Hardware Requirements

To handle the population-based nature of the Quantum-Inspired Genetic Algorithm and the extensive matrix operations for large-scale workflows, the following hardware is utilized:

- **Processor:** Intel Core i5/i7 or AMD Ryzen 5/7 (multi-core support is essential for parallel population evaluations).
- **Memory (RAM):** 16GB DDR4 (minimum 8GB) to manage high-dimensional ETC and Transfer Time matrices without page swapping.
- **Storage:** 512GB SSD (for rapid I/O operations when loading large Cybershake and Montage benchmark datasets).
- **GPU (Optional):** NVIDIA GeForce GTX 1650 or higher (can be leveraged if future iterations transition to CUDA-accelerated tensor operations).

4.1.2 Operating System

The system is designed to be cross-platform, though the primary development and testing environment is:

- **Primary OS:** Windows 11 (Version 22H2 or higher).

- **Alternative OS Support:** Ubuntu 20.04 LTS / macOS Sonoma (via Unix-based terminal execution).
- **Shell:** Windows PowerShell or Bash for executing batch simulation scripts.

4.1.3 Software Environment

The software stack focuses on scientific computing and data visualization:

- **Programming Language:** Python 3.10+ (utilized for its extensive support of mathematical libraries and readability).
- **Integrated Development Environment (IDE):** VS Code / PyCharm (equipped with Jupyter Notebook extension for iterative testing).
- **Core Libraries:**
 - **NumPy:** Essential for high-speed matrix manipulations of Q-bit amplitudes and chromosome strings.
 - **Matplotlib:** Used for generating real-time convergence curves and scheduling Gantt charts.
 - **NetworkX:** Employed for Directed Acyclic Graph (DAG) generation, topological sorting, and dependency mapping.
 - **Pandas:** Used for structured data handling of input/output CSV benchmarks.
- **Documentation Tools:** LaTeX (for formal reporting) and MS Project (for timeline management).

4.2 Detailed Design

This section converts the system requirements into a modular design blueprint. The architecture separates the **quantum engine** from the **scheduling logic** to improve reusability, scalability, and testing.

4.2.1 Workflow Ingestion and DAG Parsing Module [FR-01]

Purpose: Parse XML/TXT workflow files and build task dependencies.

- Converts workflow input into an adjacency list / DAG.
- Resolves predecessor constraints recursively.
- Computes bottom-level priority for task initialization.

4.2.2 Quantum Engine and Cumulative Measurement [FR-02, FR-03]

Purpose: Maintain quantum chromosomes and perform robust state collapse.

- Stores Q-bits as NumPy tensor: $(Population, Tasks, 2)$.
- Samples $|\beta|^2$ multiple times per task.
- Uses majority voting for stable node assignment.

4.2.3 Makespan Simulation and Fitness Module [FR-04]

Purpose: Convert task mapping into execution schedule.

- Tracks VM availability using a node vector.
- Computes EST and EFT with communication delay.
- Fitness function: $Fitness = 1/Makespan$.

4.2.4 Genetic Operator and Update Module [FR-05, FR-06]

Purpose: Evolve population toward optimal schedules.

- Rotation gate uses adaptive $\Delta\theta$.
- Task-aware crossover swaps complete task segments only.
- Prevents invalid node-ID generation.

4.2.5 Visualization and Interface Module [IR-01]

Purpose: Monitor optimization progress and visualize schedules.

- Generates Gantt charts using Matplotlib.
- Logs best makespan per generation.
- Exports convergence data to CSV.

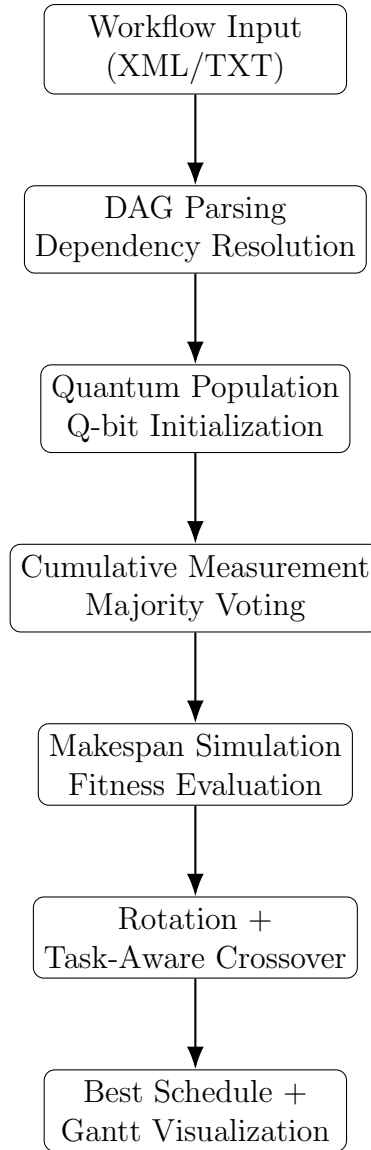


Figure 4: Detailed Design Flow of DQGA Framework

5 Implementation

The system follows a modular pipeline where each subsystem handles a dedicated phase of the quantum-inspired evolutionary scheduling process.

5.1 Workflow Ingestion & DAG Construction Module

This module converts raw workflow files into a computational DAG.

- **Technologies:** Python ElementTree, NetworkX
- **Logic:** Parses <child> XML tags, creates directed edges, and computes task bottom-level priority.

5.2 Quantum Population & Q-bit Initialization Module

This module defines the quantum search space.

- **Implementation:** 3D NumPy tensor of shape $(Population, Tasks, 2)$
- **Initialization:**

:contentReference[oaicite:0]index=0

This equal superposition ensures maximum exploration diversity.

5.3 Measurement & Fitness Evaluation Subsystem

This subsystem bridges the quantum and classical scheduling domains.

- **Cumulative Measurement:** Uses random sampling $r \in [0, 1]$ with collapse based on $|\alpha|^2$
- **Makespan Simulation:** Maps chromosomes to processors while considering communication and dependency overhead.

5.4 Quantum Rotation & Genetic Operator Module

This module drives convergence.

- **Rotation Gate:** Updates phase angle θ using lookup comparison with global best.
- **Task-Aware Crossover:** Swaps valid task sub-sequences while preserving DAG correctness.

5.5 System Installation Steps

5.5.1 Prerequisites

Ensure the following are installed:

- Python 3.9+
- Git
- GCC / Clang compiler (optional optimized extensions)

6 Test Results and Analysis

This section demonstrates the efficiency and efficacy of the proposed DQGA algorithm. The performance of the proposed algorithm is compared against four existing approaches, including the classical GA with a single crossover point, GA with a double crossover point, CQGA [29], and SQGA [30]. CQGA and SQGA are preferred for comparison because they are based on the quantum-inspired genetic algorithms. The experiments are performed on the simulated cloud computing environment implemented using Python 3.10 with high-performance numerical processing libraries, including NumPy and SciPy. All simulation runs are executed on a dedicated workstation featuring an Intel Core i7 processor operating at 3.6GHz and 16GB of RAM. The execution environment consists of fog nodes and cloud nodes. The cloud nodes consist of two types of instances, whereas the fog nodes comprise three types. The configurations of these computational nodes are given in Table 4. For the cloud node, the number of cloud instances required depends on the number of tasks that can be executed in parallel. Each computational node is associated with computing capability with an associated price that is according to the Amazon EC2 pricing scheme

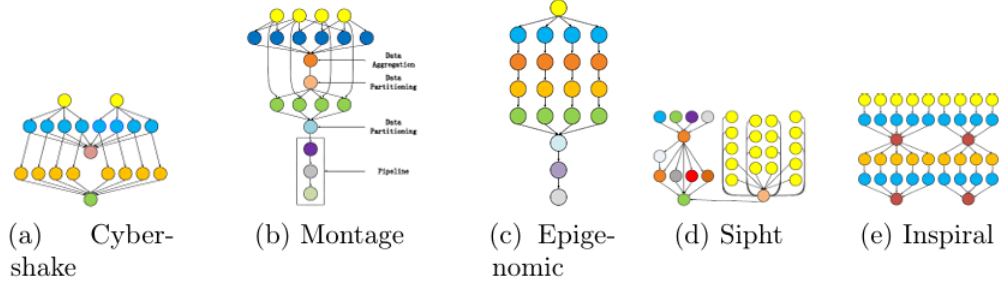


Figure 5: Structural representation of real-world workflows

Table 4: Specification of the VM instances considered

Environment	VM Instance	Cost (\$)	Cores	Memory (GB)	Bandwidth (Mbps)
Fog Computing	t3.medium	0.0416	2	8	500
	t3.large	0.0832	2	8	1000
	t3.xlarge	0.1664	4	16	1500
Cloud Computing	c4.large	0.10	2	3.75	500
	c4.xlarge	0.199	4	7.5	800

<https://aws.amazon.com/ec2/pricing/on-demand/>. The workflows considered for the experiment are generated from the Pegasus Workflow Generator [31]. Information from workflow applications is used to generate several workflows, including Montage, SIPHT, Cybershake, Epigenomic, and Inspiral (LIGO), as shown in Figure 5. These workflows are generally used to generate a custom sky mosaic, search for non-coding RNAs in bacteria, characterize earthquake hazards, perform genome sequencing, and detect and analyze gravitational waves. Further, for evaluation, workflows with 50-100 tasks are considered small, workflows with 100-500 tasks are categorized as medium-sized, and workflows with 800-1000 tasks are considered large. The experiments are performed taking various deadline factors into consideration, and these deadlines (ϕ_w) are evaluated from eq.23, where θ refers to the strictness of the deadline with the values $\theta = \{0.2, 0.4, 0.6, 0.8, 1.0\}$.

$$\phi_w = m_w + \theta * m_w \quad (23)$$

6.0.1 Performance Metrics

The performance of the several algorithms is evaluated based on the different metrics, i.e., defined as follows:

- PSR: PSR is the Planning Success Rate that defines the percentage of successful schedules generated. It can be evaluated from the eq.24.

$$PSR = \frac{SuccessfulSchedule}{TotalSchedule} * 100 \quad (24)$$

- Normalized makespan: Normalized Makespan (nm_w) is the ratio of makespan and the defined deadline. If nm_w is less than 1, the schedule is generated within the defined deadline; otherwise, the deadline is violated. The nm_w can be obtained from eq.25

$$nm_w = \frac{m_w}{\phi_w} \quad (25)$$

- Normalized Cost: Normalized Cost is obtained by dividing the total cost of execution by the maximum cost of execution (max_{cost}) as shown in eq.26

$$nc_w = \frac{tec_w}{max_{cost}} \quad (26)$$

6.0.2 Comparison of DQGA with other Classical Algorithms

DQGA has been compared with several existing algorithms, and the experimental results are shown in Figures 6, 7, and 8. Figure 6 demonstrates the evaluation of the normalized makespan of large workflows at different deadline intervals. Workflows of different sizes have been used for makespan evaluation; however, only the results from large workflows are shown here. For Cybershake (Figure 5(a)), at the strict deadline of 0.2, all the algorithms violate the given constraint. However, DQGA shows the least amount of violation. Further, when the constraints are relaxed to 0.6-1, DQGA exhibits exceptional performance compared to other algorithms. For Montage, at factor 0.4, CQGA outperforms DQGA and other algorithms. However, when the constraints are further relaxed, DQGA proves to be the best algorithm for generating a schedule that consumes the minimum makespan. For Epigenomic and LIGO, at the strict deadline of 0.2, CQGA and DQGA satisfy the constraints, while the other algorithms fail even when the constraints are relaxed. Further, when the constraints are relaxed to 0.8-1, all the algorithms generate a viable schedule. For SIPHT, as

shown in Figure 6(d), all the algorithms violate the deadline when there is a strict constraint. Whereas, the DQGA has the minimum makespan even at the deadline factor of 0.2. From the result shown in Figure 6, it can be concluded that DQGA proves to be the best algorithm with minimum deadline violations. For most cases, DQGA strives to satisfy the assigned strict constraints, while the rest of the algorithms fail and generate an infeasible schedule. The success of DQGA stems from its use of quantum bits to generate an individual population. A wide variety of populations has been generated by the generic operators that guide the schedule toward an optimal result.

Figure 6 demonstrates the result of the cost evaluation of different algorithms. The execution costs of different algorithms are evaluated over different deadline intervals. The figure shows that when deadlines are relaxed, execution cost is reduced significantly. This is because, when there is enough time for execution, the algorithms exploit the search space to generate the cheapest schedule. For Cybershake, refer to Figure 6(a), at the constraint of 0.2, SQGA exhibits the worst performance by generating an expensive schedule. CQGA generates the minimum-cost schedule, followed by DQGA. However, when the deadline is relaxed to 0.3, the DQGA shows a substantial reduction in execution cost compared to benchmark algorithms. For Montage, Epigenomic, and LIGO, the proposed SQGA outperforms all algorithms across all deadline constraints. In the case of SIPHT, the performance of CQGA and DQGA is comparable. When the constraints are from 0.3 to 0.55, DQGA shows a remarkable result, but when the constraints are relaxed to 0.6 to 0.8, CQGA achieves the minimum execution cost. When constraints are relaxed further, both algorithms exhibit similar results. On the other hand, for most of the cases, GA with single point crossover shows the worst performance, followed by SQGA.

Another comparison based on PSR has been shown in Figure 8. PSR demonstrates the percentage of viable schedules generated by each workflow. PSR is computed for workflows of all considered sizes. Figure 8(a) shows the PSR of Cybershake, where at the strict constraint of 0.2, CQGA outperforms the other algorithm by achieving 72% of success rate, followed by DQGA with 71%. When the constraints are relaxed to 1, all the algorithms achieve 100% success. GA-single point achieves 78.8%, i.e., the minimum percentage of deadline met. Similarly, for the workflows Montage, Epigenomic, SIPHT at the strict constraint of 1, all the algorithms achieve 100% success. DQGA shows an average of 82% viable schedule for Montage, 84.6% for Epigenomic

DQGA for scheduling workflows in cloud fog environment

and 89.8% for SIPHT. For LIGO, none of the algorithms generate 100% schedule even at a relaxed deadline. However, DQGA strives to achieve the highest success rate in most cases. GA-Single Point has only 80% of success at relaxed constraints. From the above analysis, it can be concluded that for most cases, DQGA performs better than the other benchmark algorithms. For most cases, DQGA shows a significant performance improvement over CQGA. The performance of Single Point GA is not up to the mark, as it fails to generate a viable schedule most of the time. Further, in most cases, it generates an expensive schedule that violates the deadline constraints. On the other hand, the performance of Double-Point GA and SQGA is comparable in terms of normalized cost evaluation.

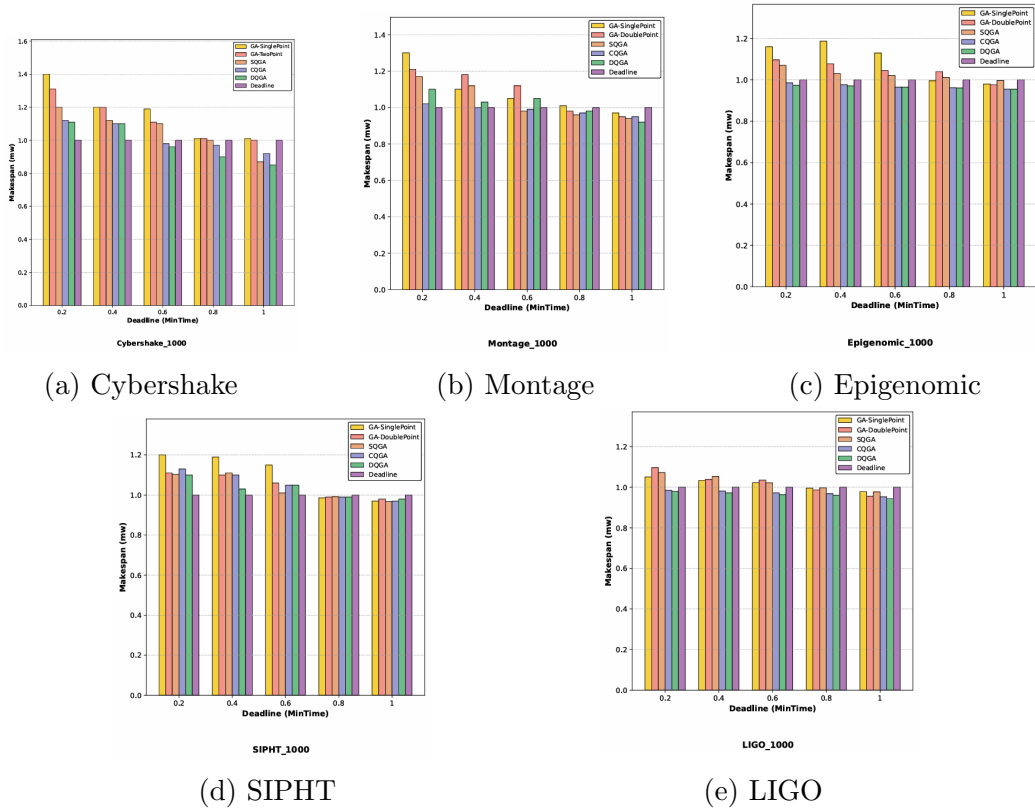


Figure 6: Makespan evaluation of Large workflows

DQGA for scheduling workflows in cloud fog environment

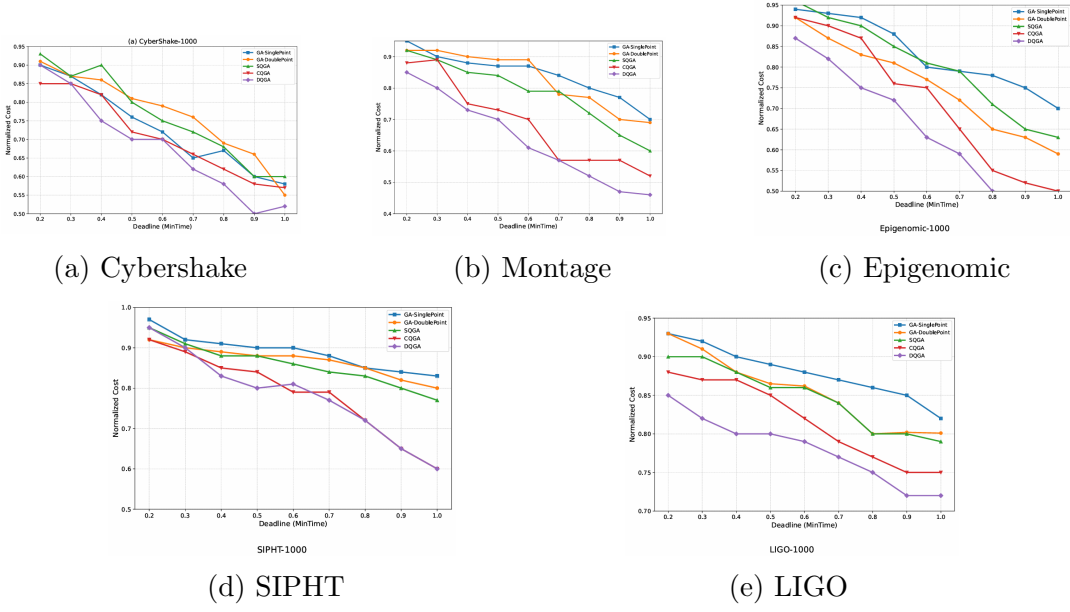


Figure 7: Normalized Cost evaluation of Large workflows

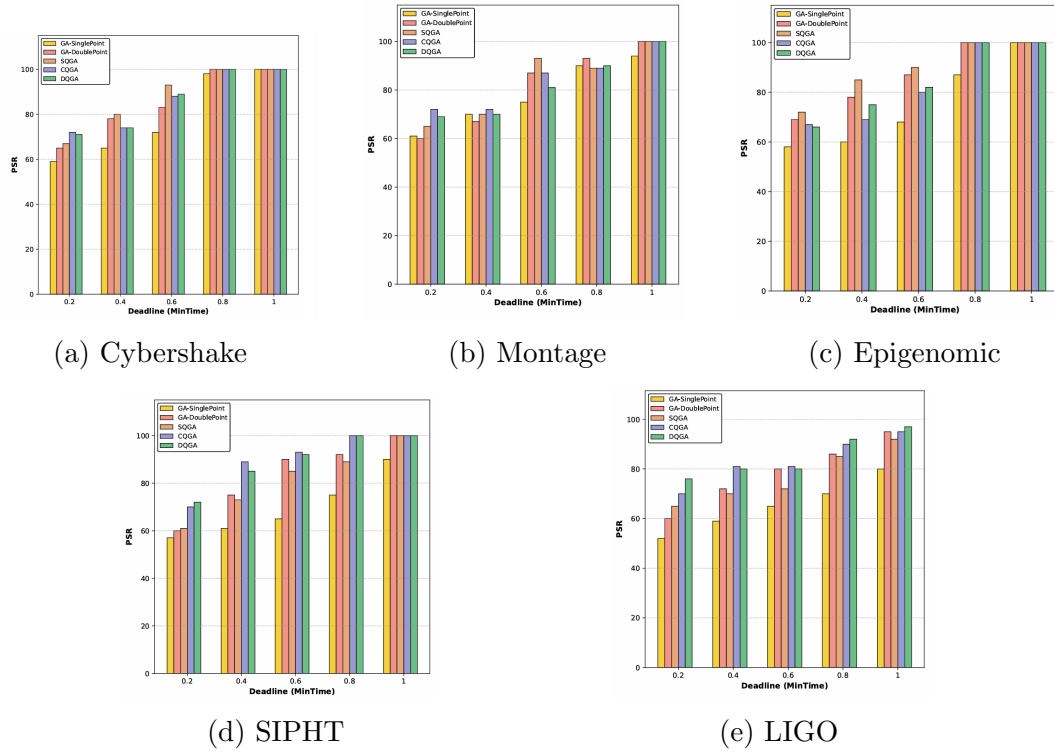


Figure 8: PSR of workflows

7 Conclusion

The execution of IoT applications across distributed cloud and fog nodes remains a significant challenge due to resource heterogeneity and dynamic workloads. Therefore, in this work, a quantum-inspired genetic algorithm with a deadline constraint is developed to address the problem. DQGA is inspired by the quantum principle and the genetic algorithm. The benefits of quantum operators and genetic operators are injected into the DQGA algorithm, which helps to generate a diverse population, adapt individuals with the best fitness value, and achieve better convergence and robustness. DQGA introduces a new cumulative-based measure function that efficiently converts a quantum probability distribution into a single classical outcome while ensuring that each basis state is selected according to its quantum mechanical probability. In addition to this, a new crossover technique named Quantum Adaptive Clustered Crossover with Rotation (QACCR) has been proposed. QACCR employs cluster-preserving crossover and an adaptive quantum rotation mechanism to balance search diversity and optimal convergence. The results show that, in most cases, DQGA outperforms existing algorithms. DQGA achieves an average of 86% of PSR. Whereas CQGA has 83.4%, SQGA has 79%, GA with Double point crossover has 76% and GA with Single Point has 73.24% of PSR.

In future research, additional investigations will be conducted to address the multi-objective workflow scheduling problem in a cloud-fog environment.

References

- [1] Mauro Tortonesi, Marco Govoni, Alessandro Morelli, Giulio Riberto, Cesare Stefanelli, and Niranjan Suri. Taming the iot data deluge: An innovative information-centric service model for fog computing applications. *Future Generation Computer Systems*, 93:888–902, 2019.
- [2] D Ravindran. Fog computing: An extended version of cloud computing. *Distrib. Comput*, 3(4), 2019.
- [3] Hindreen Rashid Abdulqadir, Subhi RM Zeebaree, Hanan M Shukur, Mohammed Mohammed Sadeeq, Baraa Wasfi Salim, Azar Abid Salih, and Shakir Fattah Kak. A study of moving from cloud computing to fog computing. *Qubahan Academic Journal*, 1(2):60–70, 2021.
- [4] Raouia Bouabdallah and Fairouz Fakhfakh. Workflow scheduling in cloud–fog computing environments: A systematic literature review. *Concurrency and Computation: Practice and Experience*, 36(28):e8304, 2024.
- [5] Samia Ijaz, Ehsan Ullah Munir, Saima Gulzar Ahmad, M Mustafa Rafique, and Omer F Rana. Energy-makespan optimization of workflow scheduling in fog–cloud computing. *Computing*, 103(9):2033–2059, 2021.
- [6] Sumit Bansal and Himanshu Aggarwal. A hybrid particle whale optimization algorithm with application to workflow scheduling in cloud–fog environment. *Decision Analytics Journal*, 9:100361, 2023.
- [7] Fuhui Wu, Qingbo Wu, and Yusong Tan. Workflow scheduling in cloud: a survey. *The Journal of Supercomputing*, 71(9):3373–3418, 2015.
- [8] Jeffrey D Ullman. Np-complete scheduling problems. *Journal of Computer and System sciences*, 10(3):384–393, 1975.
- [9] Poonam Singh, Maitreyee Dutta, and Naveen Aggarwal. A review of task scheduling based on meta-heuristics approach in cloud computing. *Knowledge and Information Systems*, 52(1):1–51, 2017.

- [10] Mirsaeid Hosseini Shirvani. A survey study on task scheduling schemes for workflow executions in cloud computing environment: classification and challenges. *The Journal of Supercomputing*, 80(7):9384–9437, 2024.
- [11] Mirsaeid Hosseini Shirvani and Kawther Kanaan Salih. A survey on meta-heuristic-based workflow scheduling algorithms running in the cloud computing platforms. *Service Oriented Computing and Applications*, pages 1–21, 2025.
- [12] Ehsan Saeedizade and Mehrdad Ashtiani. Scientific workflow scheduling algorithms in cloud environments: a comprehensive taxonomy, survey, and future directions. *Journal of Scheduling*, 28(1):1–63, 2025.
- [13] Nazik Saber Rashid and Ibrahim MI Zebari. A comprehensive review of meta-heuristic algorithms for combinatorial optimization problems. *Int. J. Sci. World*, 11(1):83–92, 2025.
- [14] Ali Mohammadzadeh, Mohammad Masdari, Farhad Soleimanian Gharehchopogh, and Ahmad Jafarian. A hybrid multi-objective metaheuristic optimization algorithm for scientific workflow scheduling. *Cluster Computing*, 24(2):1479–1503, 2021.
- [15] Amir Iranmanesh and Hamid Reza Naji. Dchg-ts: a deadline-constrained and cost-effective hybrid genetic algorithm for scientific workflow scheduling in cloud computing. *Cluster Computing*, 24(2):667–681, 2021.
- [16] Hatem Aziza and Saoussen Krichen. A hybrid genetic algorithm for scientific workflow scheduling in cloud environment. *Neural Computing and Applications*, 32(18):15263–15278, 2020.
- [17] Gyan Singh and Amit K Chaturvedi. A cost, time, energy-aware workflow scheduling using adaptive pso algorithm in a cloud–fog environment. *Computing. Archives for Informatics and Numerical Computation*, 106(10):3279–3308, 2024.
- [18] Anubhav Choudhary, Indrajeet Gupta, Vishakha Singh, and Prasanta K Jana. A gsa based hybrid algorithm for bi-objective workflow scheduling in cloud computing. *Future Generation Computer Systems*, 83:14–26, 2018.

- [19] Neetu Sharma, Puneet Garg, et al. Ant colony based optimization model for qos-based task scheduling in cloud computing environment. *Measurement: Sensors*, 24:100531, 2022.
- [20] Siyuan Tao, Yuanqing Xia, Lingjuan Ye, Ce Yan, and Runze Gao. Db-aco: A deadline-budget constrained ant colony optimization for workflow scheduling in clouds. *IEEE Transactions on Automation Science and Engineering*, 21(2):1564–1579, 2023.
- [21] Ying Xie, Yuanwei Zhu, Yeguo Wang, Yongliang Cheng, Rongbin Xu, Abubakar Sadiq Sani, Dong Yuan, and Yun Yang. A novel directional and non-local-convergent particle swarm optimization based workflow scheduling in cloud-edge environment. *Future Generation Computer Systems*, 97:361–378, 2019.
- [22] Zulfiqar Ahmad, Ali Imran Jehangiri, Mohammed Alaa Ala'anzy, Mohamed Othman, Rohaya Latip, Sardar Khaliq Uz Zaman, and Arif Iqbal Umar. Scientific workflows management and scheduling in cloud computing: taxonomy, prospects, and challenges. *IEEE Access*, 9:53491–53508, 2021.
- [23] Naela Rizvi, Dharavath Ramesh, PC Srinivasa Rao, and Koushik Mondal. Intelligent salp swarm scheduler with fitness based quasi-reflection method for scientific workflows in hybrid cloud-fog environment. *IEEE Transactions on Automation Science and Engineering*, 20(2):862–877, 2022.
- [24] Hind Mikram, Said El Kafhali, and Youssef Saadi. Hepga: A new effective hybrid algorithm for scientific workflow scheduling in cloud computing environment. *Simulation modelling practice and theory*, 130:102864, 2024.
- [25] Mohsen Khademi Dehnavi, Ali Broumandnia, Mirsaeid Hosseini Shirvani, and Iman Ahanian. A hybrid genetic-based task scheduling algorithm for cost-efficient workflow execution in heterogeneous cloud computing environment. *Cluster Computing*, 27(8):10833–10858, 2024.
- [26] Yan Wang, Jungin E Kim, and Krishnan Suresh. Opportunities and challenges of quantum computing for engineering optimization. *Journal of Computing and Information Science in Engineering*, 23(6):060817, 2023.

- [27] Junaid Ur Rehman, Muhammad Shohibul Ulum, Abdurrahman Wachid Shaffar, Amirul Adlil Hakim, Zaid Abdullah, Hayder Al-Hraishawi, Symeon Chatzinotas, Hyundong Shin, et al. Evolutionary algorithms and quantum computing: recent advances, opportunities, and challenges. *IEEE Access*, 13:16649–16670, 2025.
- [28] Dharavath Ramesh, Naela Rizvi, P. C. Srinivasa Rao, Elankovan A. Sundararajan, Koushik Mondal, Gautam Srivastava, and Lianyong Qi. Improved chemical reaction optimization with fitness-based quasi-reflection method for scheduling in hybrid cloud-fog environment. *IEEE Transactions on Network and Service Management*, 21(1):653–669, 2024.
- [29] Mehboob Hussain, Lian-Fu Wei, Amir Rehman, Muqadar Ali, Syed Muhammad Waqas, and Fakhar Abbas. Cost-aware quantum-inspired genetic algorithm for workflow scheduling in hybrid clouds. *Journal of Parallel and Distributed Computing*, 191:104920, 2024.
- [30] Raouf Belmahdi, Djamila Mechta, Saad Harous, and Abdelhak Bentaleb. Sqga: Quantum genetic algorithm-based workflow scheduling in fog-cloud computing. In *2022 International Wireless Communications and Mobile Computing (IWCMC)*, pages 131–136. IEEE, 2022.
- [31] Gideon Juve, Ann Chervenak, Ewa Deelman, Shishir Bharathi, Gaurang Mehta, and Karan Vahi. Characterizing and profiling scientific workflows. *Future generation computer systems*, 29(3):682–692, 2013.
- [32] Mohammad Masdari, Sima ValiKardan, Zahra Shahi, and Sonay Imani Azar. Towards workflow scheduling in cloud computing: a comprehensive analysis. *Journal of Network and Computer Applications*, 66:64–82, 2016.
- [33] M Menaka and KS Sendhil Kumar. Workflow scheduling in cloud environment—challenges, tools, limitations & methodologies: A review. *Measurement: Sensors*, 24:100436, 2022.
- [34] Mehdi Hosseinzadeh, Marwan Yassin Ghafour, Hawkar Kamaran Hama, Bay Vo, and Afsane Khoshnevis. Multi-objective task and workflow scheduling approaches in cloud computing: a comprehensive review: Multi-objective task and workflow scheduling approaches in cloud computing: A comprehensive review. *Journal of Grid Computing*, 18(3):327–356, 2020.

- [35] Qin-zhe Xiao, Jinghui Zhong, Liang Feng, Linbo Luo, and Jianming Lv. A cooperative coevolution hyper-heuristic framework for workflow scheduling problem. *IEEE transactions on services computing*, 15(1):150–163, 2019.
- [36] Awadh Salem Bajaher, Nor Asilah Wati Abdul Hamid, Idawaty Ahmad, and Zurina Mohd Hanapi. Predictive state-aware deep reinforcement learning with hyper-heuristic for resolving conflicting objectives in scientific workflow scheduling. *IEEE Access*, 2025.
- [37] Deafallah Alsadie. Advancements in heuristic task scheduling for iot applications in fog-cloud computing: challenges and prospects. *PeerJ Computer Science*, 10:e2128, 2024.
- [38] Faten A Saif, Rohaya Latip, Zurina Mohd Hanapi, and Kamarudin Shafinah. Multi-objective grey wolf optimizer algorithm for task scheduling in cloud-fog computing. *IEEe Access*, 11:20635–20646, 2023.
- [39] Ahmadreza Montazerolghaem, Maryam Khosravi, Fatemeh Rezaee, and Mohammad Reza Khayyambashi. An optimal workflow scheduling method in cloud-fog computing using three-objective harris-hawks algorithm. In *2022 12th International Conference on Computer and Knowledge Engineering (ICCKE)*, pages 300–306. IEEE, 2022.
- [40] Navid Khaledian, Marcus Voelp, Sadoon Azizi, and Mirsaeid Hosseini Shirvani. Ai-based & heuristic workflow scheduling in cloud and fog computing: a systematic review. *Cluster Computing*, 27(8):10265–10298, 2024.
- [41] Samia Ijaz, Saima Gulzar Ahmad, Kashif Ayyub, Ehsan Ullah Munir, and Naeem Ramzan. Energy-efficient time and cost constraint scheduling algorithm using improved multi-objective differential evolution in fog computing: S. ijaz et al. *The Journal of Supercomputing*, 81(1):116, 2025.
- [42] Gyan Singh and Amit K Chaturvedi. Hybrid modified particle swarm optimization with genetic algorithm (ga) based workflow scheduling in cloud-fog environment for multi-objective optimization. *Cluster Computing*, 27(2):1947–1964, 2024.

- [43] Munish Bhatia, Sandeep K Sood, and Simranpreet Kaur. Quantum-based predictive fog scheduler for iot applications. *Computers in Industry*, 111:51–67, 2019.
- [44] Reyhane Ghafari and Najme Mansouri. Deadline-aware and priority-driven scheduling for iot tasks in fog-cloud systems using reinforcement learning. *Cluster Computing*, 28(14):920, 2025.
- [45] Jinhong Luo, Yunfan Zhou, Xijun Li, Mingxuan Yuan, Jianguo Yao, and Jia Zeng. Learning to optimize dag scheduling in heterogeneous environment. *arXiv preprint arXiv:2103.06980*, 2021.

APPENDIX A - Prototypes

A Prototype Features

The proposed Deadline-Based Quantum Inspired Genetic Algorithm (DQGA) prototype was implemented in Python and consists of the following major modules.

1. Workflow input parser for ETC and Task Dependency (TT) files.
2. Task graph construction from dependency matrix.
3. Workflow DAG visualization.
4. Quantum chromosome initialization.
5. Cumulative statistical measurement for quantum state observation.
6. Adaptive quantum rotation gate.
7. Makespan and execution cost evaluation.
8. Resource allocation and scheduling.

Figures 9–14 show important implementation modules of the proposed system.

B Filtered Source Code Showing Prototype Features

B.1 Workflow Input Processing

```
def load_etc(path):
    vals = []
    with open(path, "r") as f:
        lines = [l.strip() for l in f if l.strip()]

    # First Line: T N
    T, N = map(int, lines[0].split())

    # Remaining: T*N values in a column
    for v in lines[1:]:
        vals.append(float(v))

    etc = np.array(vals).reshape(T, N)
    return etc
```

```
def load_tt(path):
    with open(path, "r") as f:
        lines = [l.strip() for l in f if l.strip()]

    n = int(lines[0])
    data = []
    for i in range(1, 1+n):
        row = list(map(float, lines[i].split()))
        data.append(row)

    return np.array(data)
```

Figure 9: Workflow input parser for ETC and TT files

The module loads the ETC matrix containing execution times and the TT matrix representing task dependencies. These files are converted into NumPy matrices for further scheduling operations.

B.2 Task Graph Construction and DAG Visualization

```
def build_tasks(tt):
    T = tt.shape[0]
    tasks = [Task(i) for i in range(T)]

    for i in range(T):
        for j in range(T):
            if tt[i][j] > 0:
                tasks[j].add_parent(i, tt[i][j])

    return tasks

def plot_dag(tasks, name):
    G = nx.DiGraph()

    for t in tasks:
        G.add_node(t.id)

    for t in tasks:
        for (p, data) in t.parents:
            G.add_edge(p, t.id, weight=data)

    plt.figure(figsize=(10, 6))
    pos = nx.spring_layout(G, seed=42)

    nx.draw_networkx_nodes(G, pos, node_color="lightblue", node_size=700)
    nx.draw_networkx_labels(G, pos)
    nx.draw_networkx_edges(G, pos, arrowstyle="->", arrowsize=20)

    labels = nx.get_edge_attributes(G, 'weight')
    nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)

    plt.title(f"{name} Workflow DAG")
    plt.axis("off")
    plt.show()
```

Figure 10: Task construction and DAG generation module

This module creates task objects from the dependency matrix and generates the workflow Directed Acyclic Graph (DAG) using the NetworkX library.

B.3 Quantum-Inspired Cumulative Measurement

```

def dqga_approach(bpn, T, qchrom, iter = 15):
    final_bits = []
    num_states = 2**bpn
    for t in range(T):
        qinspired_amp = qchrom[t*bpn:(t+1)*bpn]
        #Convert amplitudes into probability
        state_prob = [1.0]*num_states
        for i in range(num_states):
            for bit_pos in range(bpn):
                alpha, beta = qinspired_amp[bit_pos]
                if (i & (1 << (bpn - 1 - bit_pos))) :
                    state_prob[i] *= beta**2
                else:
                    state_prob[i] *= alpha**2
        cf = []
        curr = 0
        for p in state_prob:
            curr += p
            cf.append(curr)
        #Statistical Sampling
        samples = []
        for _ in range(iter):
            r = random.random()
            if cf[-1] > 0:
                r *= cf[-1]
            for start_idx, threshold in enumerate(cf):
                if r < threshold:
                    samples.append(start_idx)
                    break
        if not samples:
            winner_state = 0
        else:
            winner_state = Counter(samples).most_common(1)[0][0]
            winner_bits = [int(b) for b in bin(winner_state)[2:].zfill(bpn)]
            final_bits.extend(winner_bits)
    return final_bits

```

Figure 11: Deadline-based Quantum Inspired Genetic Algorithm (DQGA)

The algorithm converts quantum amplitudes into probability distributions, performs cumulative statistical sampling multiple times, and determines the final observed state using majority voting.

B.4 Adaptive Quantum Rotation Gate

```
def get_rotation_angle(x, b, fit_x, fit_best, a, beta, DELTA):
    condition = (fit_x >= fit_best) # current makespan i worse or equal

    #By table 1 logic
    if x == 0 and b == 0:
        return 0
    elif x == 0 and b == 1:
        if not condition: #f(x) < f(b)
            s = 1 if a*beta > 0 else -1 if a*beta < 0 else 0 if a == 0 else 1
        else:
            s = -1 if a*beta > 0 else 1 if a*beta < 0 else 1 if a == 0 else 0
        return s*DELTA
    elif x == 1 and b == 0:
        if not condition:
            s = -1 if a*beta > 0 else 1 if a*beta < 0 else 1 if a == 0 else 0
        else:
            s = 1 if a*beta > 0 else -1 if a*beta < 0 else 0 if a == 0 else 1
        return s*DELTA
    elif x == 1 and b == 1:
        return 0
    return 0
```

Figure 12: Adaptive quantum rotation gate implementation

The adaptive rotation gate updates the quantum chromosome according to the fitness comparison between the current solution and the global best solution.

B.5 Makespan and Cost Evaluation

```

def compute_makespan(tasks, nodes, etc, assignment):
    T = len(tasks)
    EST = [0]*T
    EFT = [0]*T
    #total_cost = 0.0 #initializing the cost tracker

    # Topological order
    indeg = {t.id: 0 for t in tasks}
    edges = {t.id: [] for t in tasks}

    for t in tasks:
        for (p, d) in t.parents:
            indeg[t.id] += 1
            edges[p].append((t.id, d))

    Q = [i for i in indeg if indeg[i] == 0]
    topo = []
    while Q:
        u = Q.pop(0)
        topo.append(u)
        for (v,_) in edges[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                Q.append(v)

    for tid in topo:
        task = tasks[tid]
        node_idx = assignment[tid]

        start = 0
        for (p, data) in task.parents:
            p_node = assignment[p]
            finish_p = EFT[p]

            if p_node == node_idx:
                trans = 0

```

Figure 13: Makespan computation (Part I)

DQGA for scheduling workflows in cloud fog environment

```
task = tasks[tid]
node_idx = assignment[tid]

start = 0
for (p, data) in task.parents:
    p_node = assignment[p]
    finish_p = EFT[p]

    if p_node == node_idx:
        trans = 0
    else:
        bw = min(nodes[node_idx].bw, nodes[p_node].bw)
        trans = data / (bw/8)

    start = max(start, finish_p + trans)

dur = etc[tid][node_idx]
EST[tid] = start
EFT[tid] = start + dur

cmpw = 0.0 #Computational Cost
ccpw = 0.0 #Communication Cost
for i in range(T):
    n_idx = assignment[i]
    cmpw += (EFT[i] - EST[i])*nodes[n_idx].cpr #Execution duration * node computation price

    for (pid, data) in tasks[i].parents:
        p_node = assignment[pid]
        if n_idx != p_node:
            trans_time = data/(min(nodes[n_idx].bw,nodes[p_node].bw)/8)
            ccpw += trans_time*nodes[p_node].ctr
total_cost = cmpw + ccpw
makespan = max(EFT)
schedule_data = [(EST[i], total_cost, EFT[i], assignment[i]) for i in range(T)]

return makespan, total_cost, schedule_data
```

Figure 14: Makespan computation (Part II)

The scheduling module performs topological traversal of workflow tasks, computes the Earliest Start Time (EST), Earliest Finish Time (EFT), communication delay, computational cost, communication cost, total execution cost and overall workflow makespan.

C Software Requirements

Table 5: Software Requirements

Operating System	Windows 10/11 or Ubuntu Linux
Programming Language	Python 3.10 or above
IDE	Visual Studio Code / PyCharm
Libraries	NumPy, NetworkX, Matplotlib, Tabulate
Package Manager	pip

D Hardware Requirements

Table 6: Hardware Requirements

Processor	Intel Core i5 or above
RAM	Minimum 8 GB
Storage	10 GB free space
Operating System	Windows/Linux

E Compilation and Execution Steps

The proposed prototype was executed using Python. The following steps were followed during compilation and execution.

1. Install Python (Version 3.10 or higher).
2. Install the required libraries using:

```

pip install numpy
pip install matplotlib
pip install networkx
pip install tabulate

```

3. Download or clone the project source code.
4. Place the workflow datasets (.etc and .tt files) inside the **datasets** directory.
5. Open the project in Visual Studio Code or PyCharm.

DQGA for scheduling workflows in cloud fog environment

6. Execute the main program using

```
python main.py
```

7. The program performs the following operations:

- (a) Reads ETC and TT files.
- (b) Builds the workflow DAG.
- (c) Initializes quantum chromosomes.
- (d) Executes the DQGA scheduling algorithm.
- (e) Computes makespan and execution cost.
- (f) Generates workflow DAG and Gantt chart.
- (g) Displays scheduling statistics.

8. The output consists of

- Workflow DAG
- Task scheduling table
- Gantt chart
- Makespan
- Computational cost
- Communication cost
- Total execution cost

F Project Directory Structure

Project/

```
main.py  
scheduler.py  
dqga.py  
task.py
```

```
node.py
datasets/
    workflow.etc
    workflow.tt
output/
    dag.png
    gantt.png
    results.csv
requirements.txt
```

APPENDIX B - Paper publications (optional)

The following research paper has been submitted to an international journal and is currently under review.

1. Aniruddha Sharma, [Co-author(s)], “Deadline-Based Quantum-Inspired Genetic Algorithm for Scheduling Workflows in Cloud-Fog Environment” Submitted to *Computing*, Under Review, 2026.
2. Aniruddha Sharma, [Co-author(s)], “Deadline-Based Quantum-Inspired Genetic Algorithm for Scheduling Workflows in Cloud-Fog Environment” Submitted to *Journal of Systems Architecture*, Under Review, 2026.
3. Aniruddha Sharma, [Co-author(s)], “Deadline-Based Quantum-Inspired Genetic Algorithm for Scheduling Workflows in Cloud-Fog Environment” Submitted to *Sustainable Computing: Informatics and Systems* , Under Review 2026.
4. Aniruddha Sharma, [Co-author(s)], “Deadline-Based Quantum-Inspired Genetic Algorithm for Scheduling Workflows in Cloud-Fog Environment” Submitted to *Ad Hoc Networks*, Under Review 2026.
5. Aniruddha Sharma, [Co-author(s)], “Deadline-Based Quantum-Inspired Genetic Algorithm for Scheduling Workflows in Cloud-Fog Environment” Submitted to *Journal of Systems and Software*, Under Review 2026.

DQGA for scheduling workflows in cloud fog environment

6. Aniruddha Sharma, [Co-author(s)], “Deadline-Based Quantum-Inspired Genetic Algorithm for Scheduling Workflows in Cloud-Fog Environment” Submitted to *Simulation Modelling Practice and Theory.*, Under Review 2026.

Status: The manuscript has been submitted and is currently under peer review. At the time of submission of this project report, the paper has not yet been accepted for publication.

Submission Evidence



Figure 15: Co-author confirmation email from Journal of Systems Architecture.

DQGA for scheduling workflows in cloud fog environment

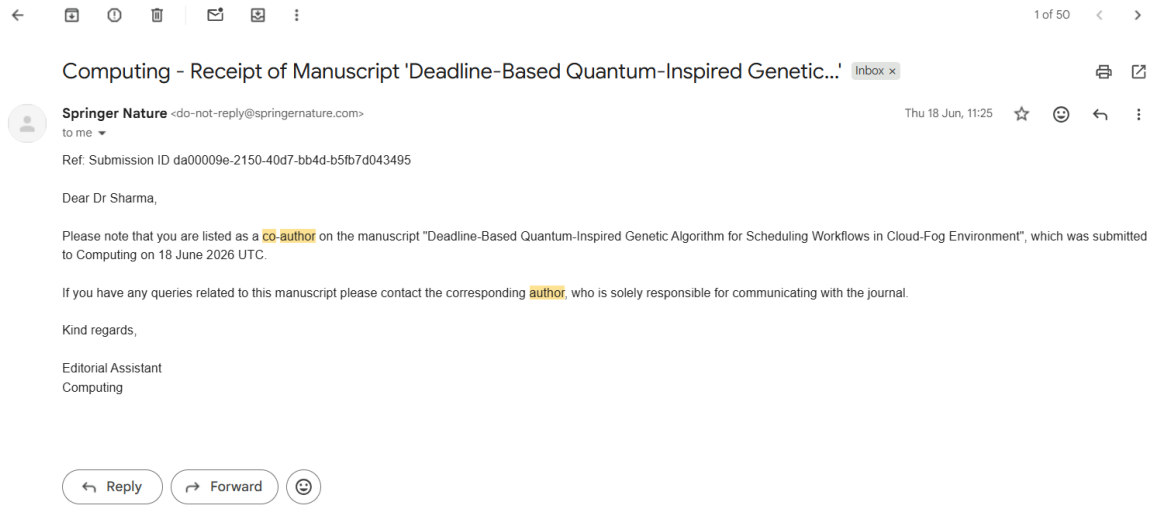


Figure 16: Submission acknowledgement from Computing (Springer Nature).

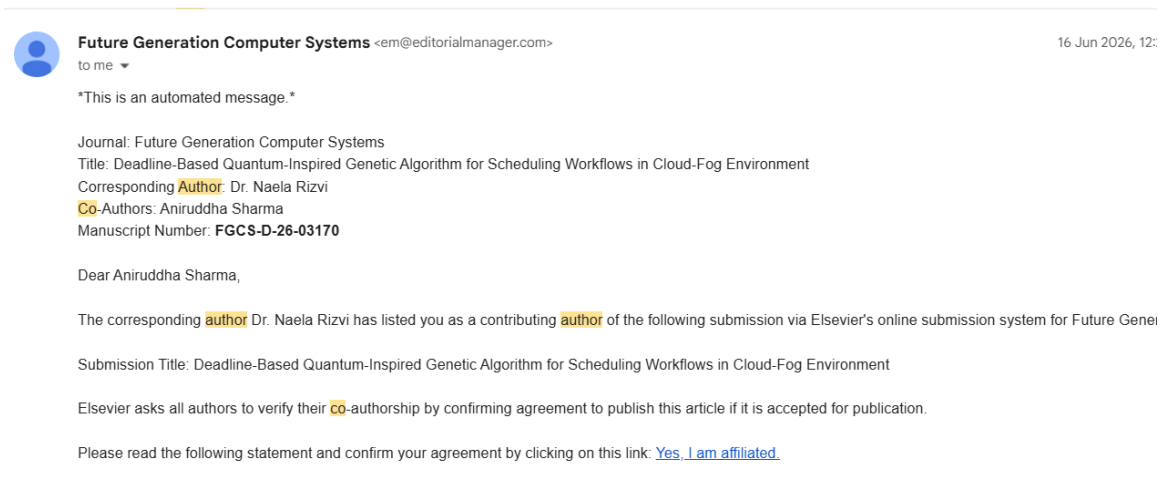


Figure 17: Co-author confirmation email from Future Generation Computer Systems.

DQGA for scheduling workflows in cloud fog environment

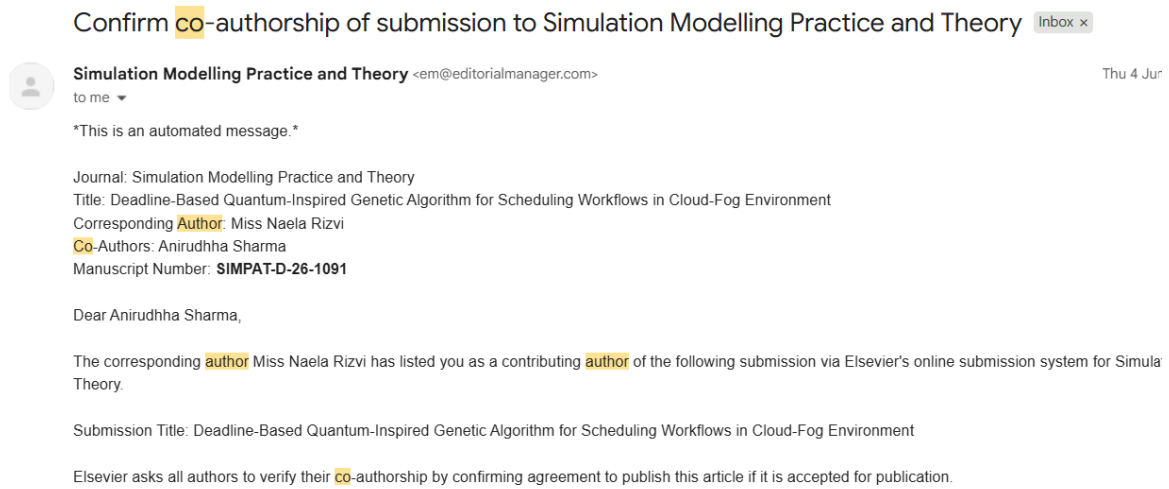


Figure 18: Co-author confirmation email from Simulation Modelling Practice and Theory.

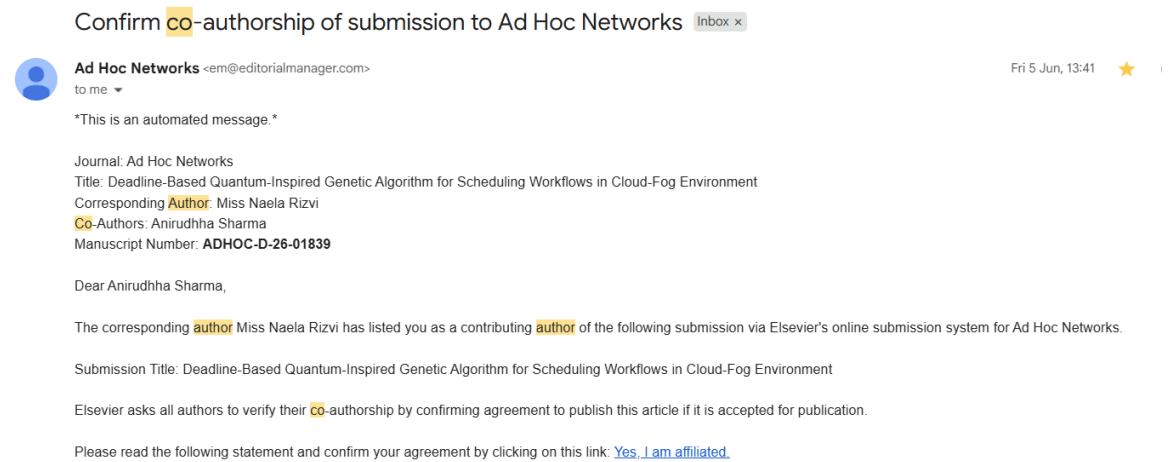


Figure 19: Co-author confirmation email from Ad Hoc Networks.

DQGA for scheduling workflows in cloud fog environment

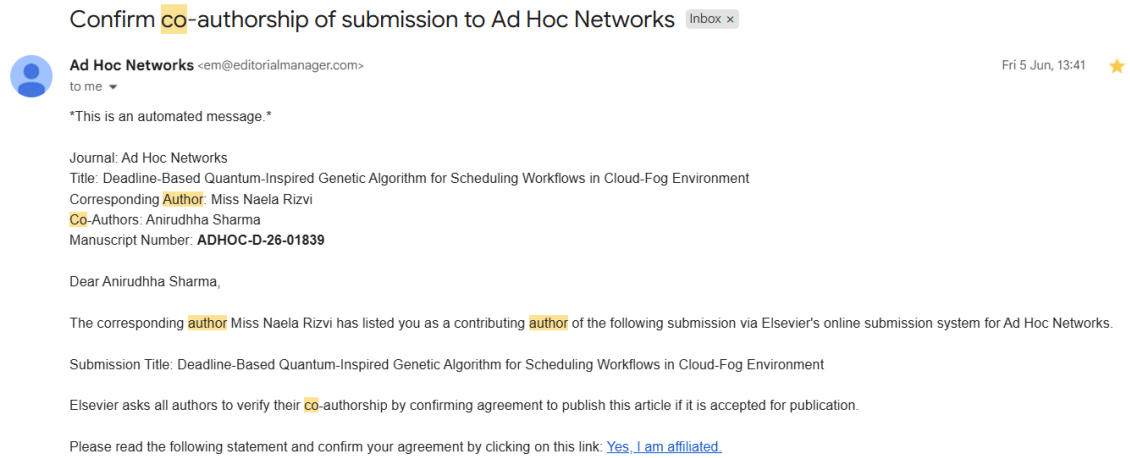


Figure 20: Co-author confirmation email from Journal of Systems and Software.